

Rock, Paper, Scissors

Overview

1. Purpose and patterns

This agent-based model simulates a competition of rock, paper, scissors, analyzing how initial population distributions and vision radiuses influence win outcomes.

2. Entities, state variables, and scales

The model consists of three agent breeds representing teams: rock, paper, or scissors. State variables consist of agent type and their position in the world. The environment is wrapped both horizontally and vertically, ranging from -20 to 20 on both axes. So, agents that move off one edge reappear on the opposite side. Time is measured in discrete ticks, and the simulation runs until one agent type dominates or until 1500 ticks are reached.

3. Process overview and scheduling

At each tick, agents move forward within the environment. When two different breed agents collide, an interaction occurs where the losing agent is converted into the winning agent's type based on rock–paper–scissors relationships of predator and prey. If the vision feature is enabled, scissors agents can detect nearby agents within a specified radius and will either chase prey or avoid predators. Speeds and sizes between agent types are equal. The simulation continues until one population dominates or the time limit is reached (1500 ticks).

Design Concepts

4. Design concepts

This model features stochasticity, sensing, and prediction. Turtle agents spawn across the world randomly, in different directions, during the set up procedure. And when the scissor vision feature is turned on, the scissors team gains an adjustable vision radius, which influences its movement patterns. Seeing its prey (paper), it will target/chase it. Seeing its predator (rock), it will flee/turn another direction.

Details

5. Initialization

Agents are initialized with user-defined population sizes for each type (rock, paper, scissors). The default values are a 20 starting count for each agent breed type and a scissor vision radius of zero/off. They are randomly distributed across the environment at the start of the simulation.

6. Input data

No external datasets are used. Model parameters such as population size and vision radius are controlled through sliders and a switch.

7. Submodels

For movement, a submodel determines that all agents move forward randomly each tick by default. If vision is enabled (for scissors), scissors agents adapt movement by chasing prey or avoiding predators.

For competing interactions:

- Rock + Scissors → Scissors becomes Rock
- Scissors + Paper → Paper becomes Scissors
- Paper + Rock → Rock becomes Paper

When an agent loses an interaction, it changes its type to match the winning agent. This includes updating both its internal state (breed type) and visual representation (color). This is a set of conversion procedure submodels, called become-rock, become-paper, and become-scissors.

Another submodel updates the count of agents belonging to each type (rock, paper, scissors) for every tick. These counts become visualized in the plot widget on the interface tab.

Lastly, a submodel, called check-winner, has two conditions which determines whether the simulation ends: if one agent type reaches full dominance (all agents are the same type), or if the simulation reaches a maximum of 1500 ticks without a dominant type (timeout condition).